

Progress Report

Day 6 — JWT Authentication Implementation

Task Management MERN Stack Platform

Shafin Nigamana | 24 May 2026

1. Day-6 Objective

The goal for Day 6 was to implement JWT-based authentication, secure user passwords using bcrypt hashing, and protect backend routes using authentication middleware.

2. Tasks Completed

User Registration

- Implemented POST /api/auth/register endpoint
- Added user creation flow using MongoDB
- Configured bcrypt password hashing before database storage
- Generated JWT token after successful registration

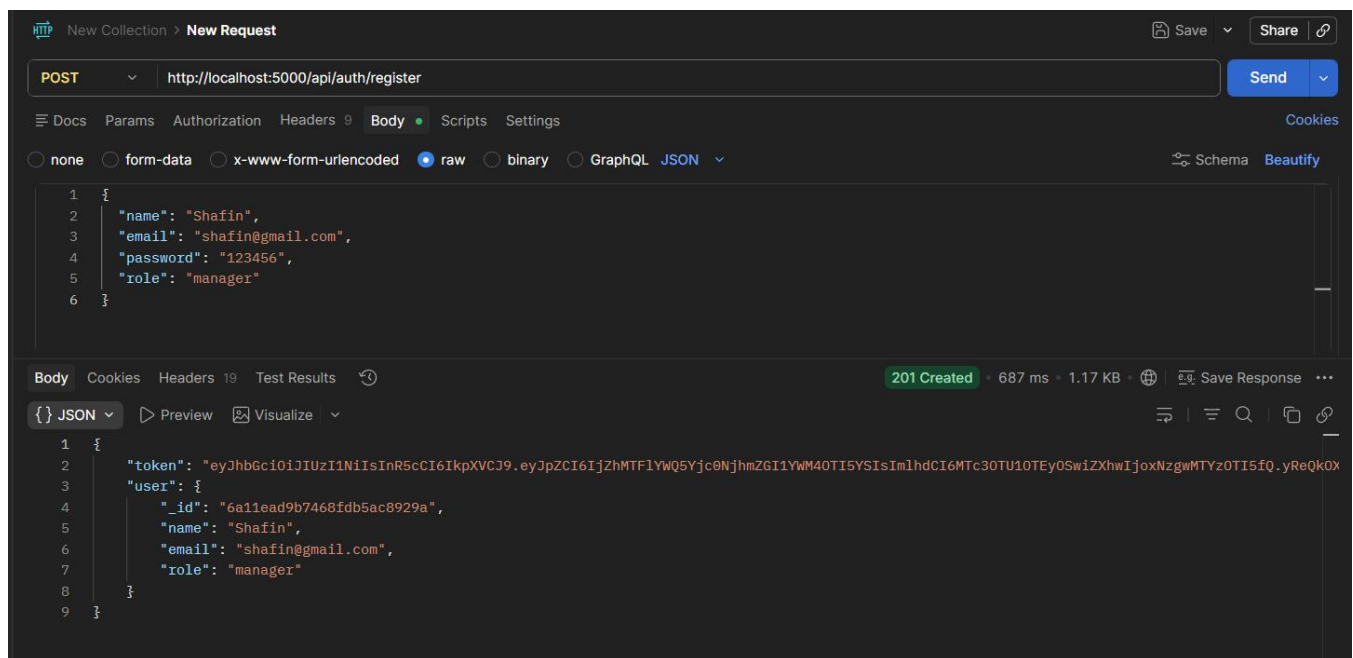


Figure 2.1 — User registration endpoint tested successfully.

User Login

- Implemented POST /api/auth/login endpoint
- Verified user credentials using bcrypt password comparison
- Generated JWT token after successful login
- Returned authenticated user information without passwordHash

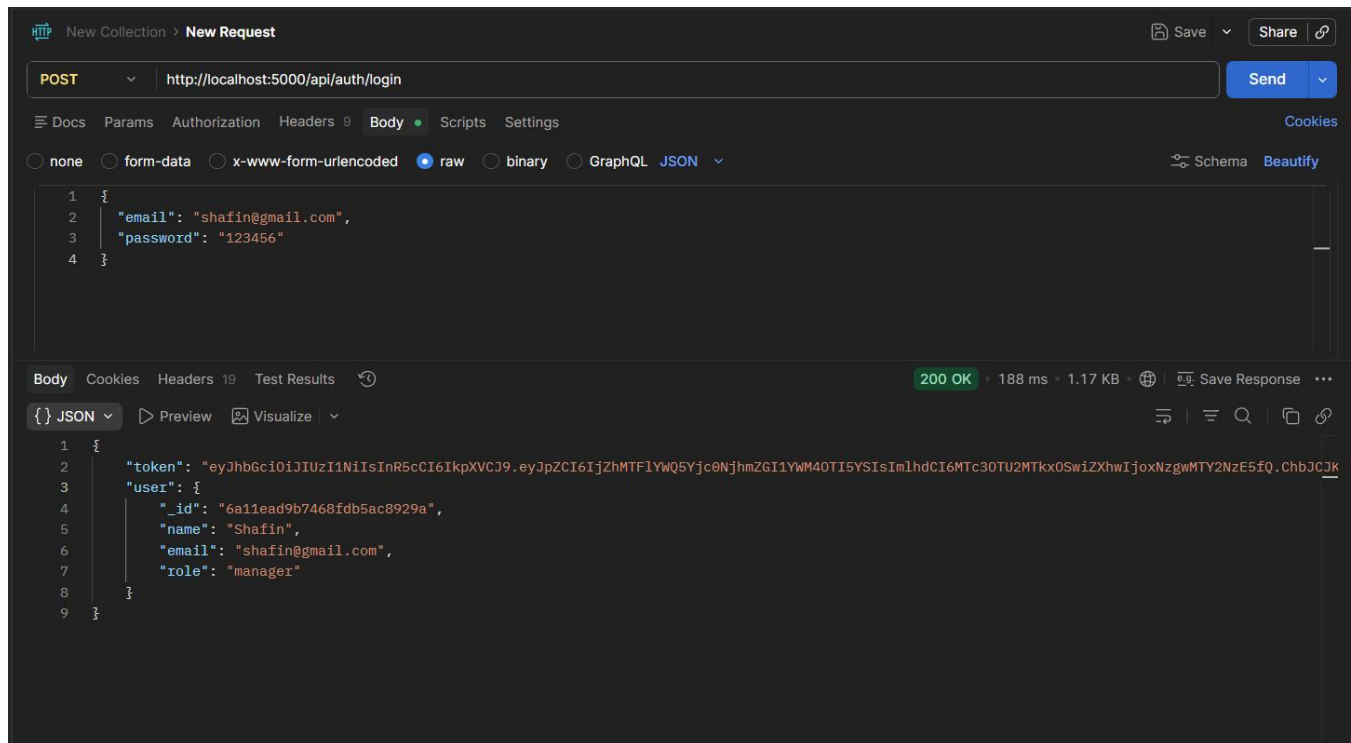
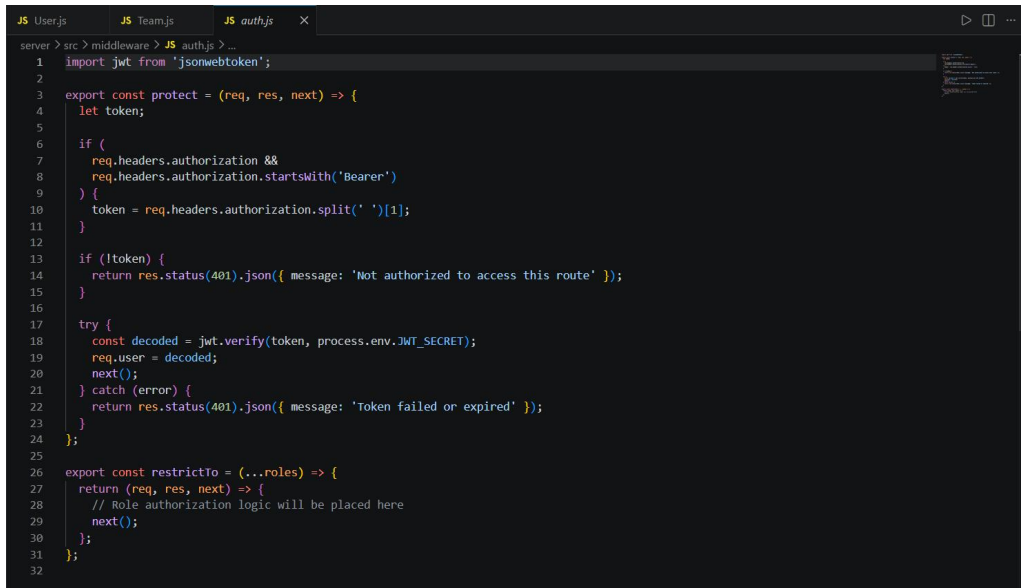


Figure 2.2 — User login endpoint returning JWT token.

JWT Authentication Middleware

- Implemented authentication middleware using jsonwebtoken
- Configured Bearer token extraction from request headers
- Added JWT verification using jwt.verify()
- Attached authenticated user information to req.user



```
server > src > middleware > JS auth.js > ...
1 import jwt from 'jsonwebtoken';
2
3 export const protect = (req, res, next) => {
4   let token;
5
6   if (
7     req.headers.authorization &&
8     req.headers.authorization.startsWith('Bearer')
9   ) {
10    token = req.headers.authorization.split(' ')[1];
11  }
12
13  if (!token) {
14    return res.status(401).json({ message: 'Not authorized to access this route' });
15  }
16
17  try {
18    const decoded = jwt.verify(token, process.env.JWT_SECRET);
19    req.user = decoded;
20    next();
21  } catch (error) {
22    return res.status(401).json({ message: 'Token failed or expired' });
23  }
24 };
25
26 export const restrictTo = (...roles) => {
27   return (req, res, next) => {
28     // Role authorization logic will be placed here
29     next();
30   };
31 };
32
```

Figure 2.3 — JWT authentication middleware integrated into backend routes.

Protected Route Implementation

- Implemented protected GET /api/auth/me route
- Verified authenticated user access using Bearer token
- Restricted unauthenticated requests using middleware protection

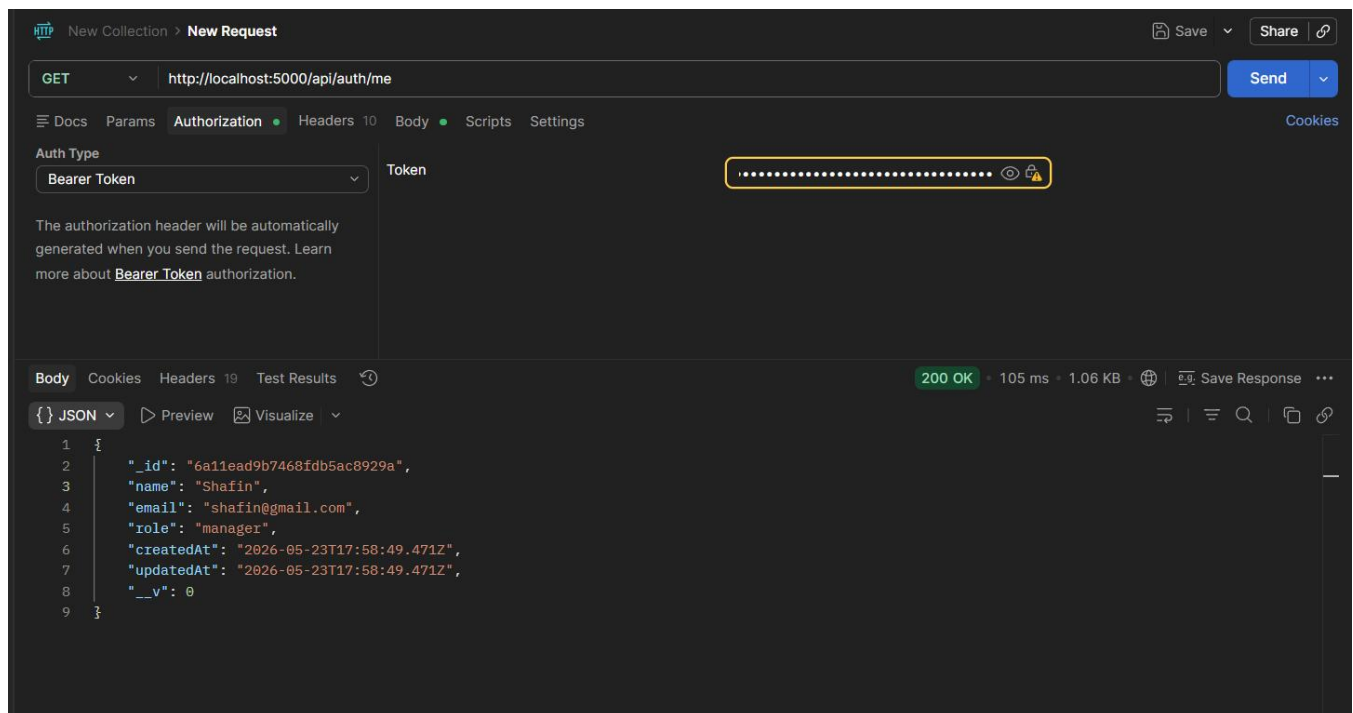


Figure 2.4 — Protected /me route returning authenticated user data.

Unauthorized Access Verification

- Tested protected route access without JWT token
- Verified backend returns unauthorized response correctly

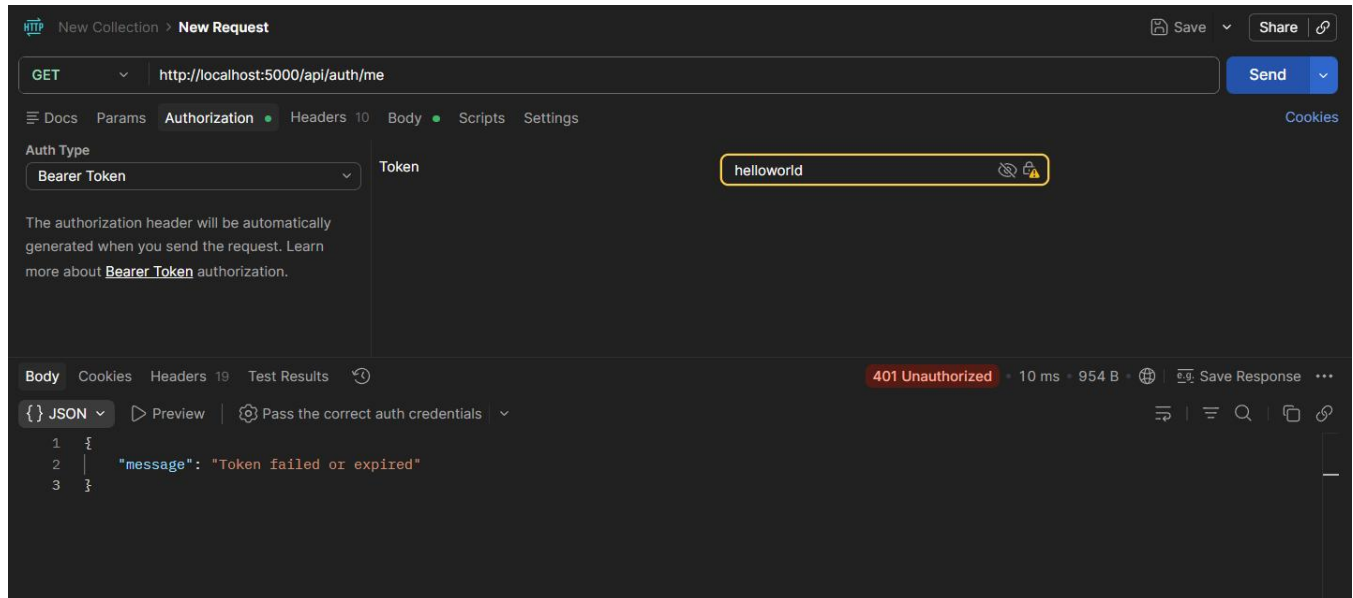


Figure 2.5 — Unauthorized request blocked by authentication middleware.

Password Hash Verification

- Verified passwords are stored as bcrypt hashes inside MongoDB Atlas
- Confirmed plain text passwords are not stored in the database

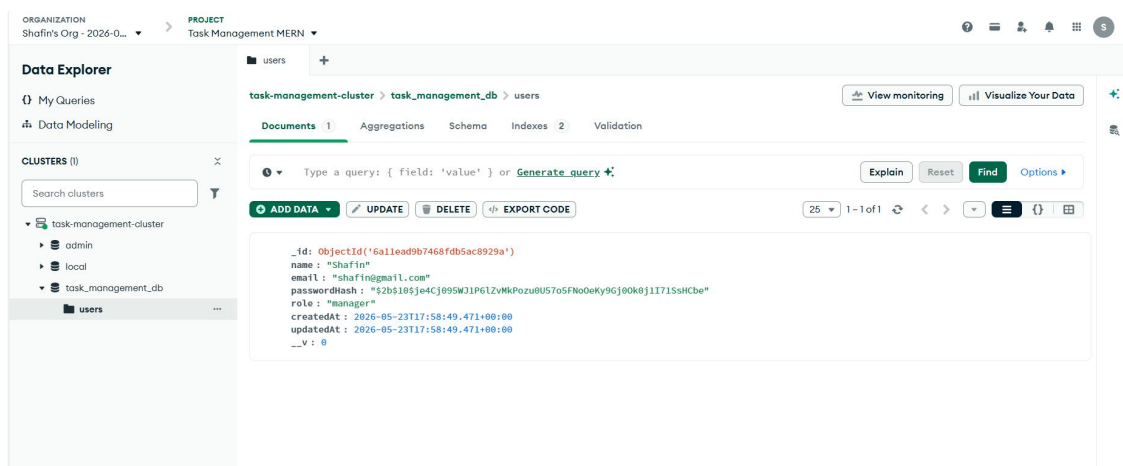
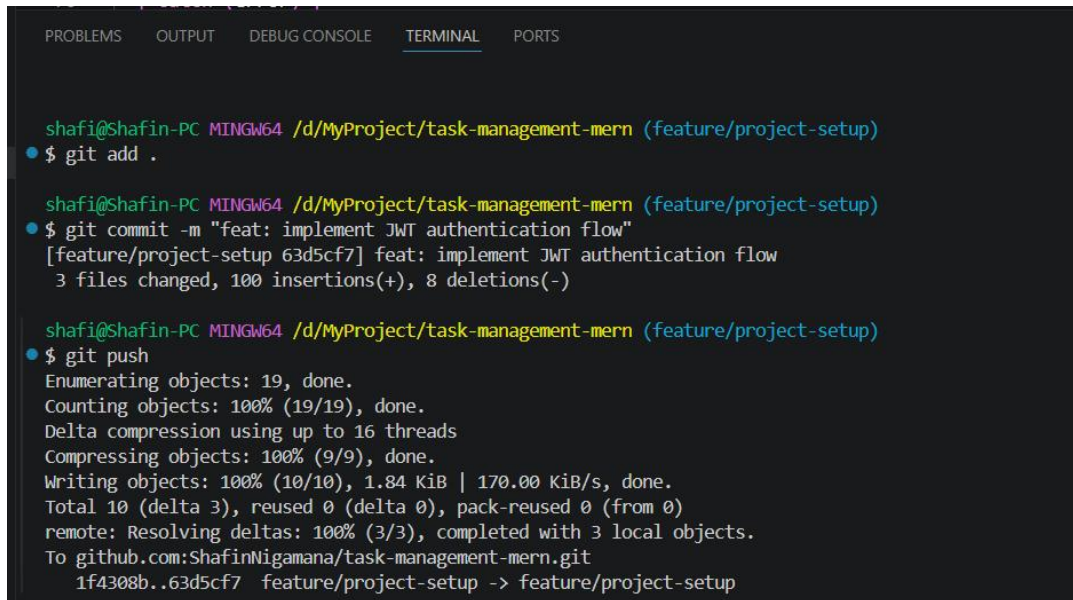


Figure 2.6 — Password stored as bcrypt hash inside MongoDB collection.

Repository Update

- Committed authentication implementation changes using Git workflow
- Pushed latest backend authentication updates to GitHub repository



```
shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
$ git add .

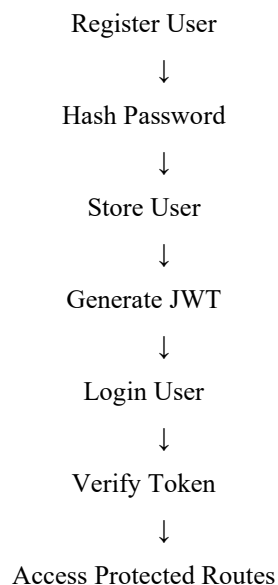
shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
$ git commit -m "feat: implement JWT authentication flow"
[feature/project-setup 63d5cf7] feat: implement JWT authentication flow
3 files changed, 100 insertions(+), 8 deletions(-)

shafi@Shafin-PC MINGW64 /d/MyProject/task-management-mern (feature/project-setup)
$ git push
Enumerating objects: 19, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 16 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 1.84 KiB | 170.00 KiB/s, done.
Total 10 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To github.com:ShafinNigamana/task-management-mern.git
1f4308b..63d5cf7 feature/project-setup -> feature/project-setup
```

Figure 2.7 — Day 6 JWT authentication implementation pushed to GitHub.

3. Current Authentication Flow

Current authentication flow:



4. Challenges Faced

- Ensuring passwordHash is stored securely instead of plain text passwords
- Configuring middleware to correctly extract and verify Bearer tokens from request headers
- Protecting backend routes while keeping the implementation structure minimal and clean

5. Next Steps (Day-7)

- Begin Team CRUD endpoint implementation
- Add manager-only authorization middleware
- Prepare backend role-based access control flow
- Continue API testing using Postman

6. Conclusion

Day 6 focused on implementing backend authentication using JWT and bcrypt. User registration, login, token verification, and protected route handling were implemented successfully. Password hashing and authentication middleware were verified using Postman and MongoDB Atlas.